מטלת בית: Takam Search

חיפוש טוב יותר בהוראות תכ"ם: בעיה, פתרון, ארכיטקטורה, AI ובדיקות

נתן מסיקה · 11.11.2026 האתר: `takam-search.vercel.app` הקוד: `github.com/nathmsi/takam-search`

1 תקציר

הבעיה: אדם שיש לו מצב קונקרטי לא מצליח להגיע בוודאות להוראת תכ"ם שחלה עליו, כי החיפוש באתר הוא "מכיל" פשוט על כותרת, תיאור, מילות מפתח ומספר. **הפתרון:** מסד נתונים מקביל בבעלותנו, ו-API חיפוש אמיתי מעליו שנכתב ב-SQL. **בסוף הוספתי AI:** כפתור בכותרת העליונה שמדליק חיפוש לפי משמעות (embeddings ב-`pgvector`). הכול ב-`TypeScript`, ב-`monorepo` אחד, עם בדיקות בכל שכבה וצנרת אחת שפורסת רק אחרי שהכול עבר.

2 הבעיות שמצאתי באתר תכ"ם

אתר הוראות תכ"ם (`takam.mof.gov.il`) הוא המקום שבו עובדי משרדי הממשלה מחפשים את ההנחיות המחייבות של החשב הכללי. השתמשתי באתר, קראתי את הקוד שלו וקראתי ל-API הציבורי שלו (1,126 מסמכים, 114 פרקים):

- **החיפוש לא מוצא:** יחיד ורבים הן שתי מילים שונות, ושאלה לא מחזירה כלום (הבעיה המרכזית, בפרק הבא).
- **דף שנשבר כשאין תוצאות:** הכתובת הופכת ל-`/document/undefined` וחלונית הפרטים נשארת ריקה.
- **דגל התוקף הפוך:** `isvalid` הוא `true` דווקא ב-51 מסמכים שתוקפם פג, והאתר מסתיר זאת בקוד.
- **פרק שלם חסר בסינון:** פרק 10, "ניהול מלאי ומצאי" (21 הוראות), לא מופיע ב-`/api/Chapter`.
- **11 הוראות לא מוצגות בעץ**, וחיפוש לפי המספר שלהן לא מוצא אותן (ל-7 מהן התיאור אומר שאוחדו, כנראה במכוון).
- **סינון התגיות מת:** `/api/Tag` ריק.
- **ארבע בעיות באופן שבו האתר טוען את הנתונים** (בטבלה שלמטה): קריאה אחת שמביאה הכול, בלי מטמון, שלוש קריאות בשרשרת לכל הוראה, וקריאות שהתשובה שלהן ריקה.
- **התנהגויות API תמוהות:** מזהה קובץ שגוי מחזיר 503 עם HTML, כתובת לא קיימת מחזירה 200 עם דף הבית, וכותרות CORS שמכריזות על פעולות כתיבה.

ארבע הבעיות באופן שבו האתר טוען נתונים

נמדד ב-11.11.2026 עם `curl` ועם לשונית הרשת בדפדפן. אלו תצפיות על התנהגות ה-API, לא על הקוד שלו (אותו לא ראיתי).

הבעיה	מה קורה	למה זו בעיה	מה עשיתי
1. קריאה אחת מביאה הכול	`GET /api/Document` מחזיר את כל 1,126 המסמכים: 875,738 בתים (75KB דחוסים). פרמטר לא משנה כלום: `?search=עיקול` מחזיר בדיוק אותם בתים.	השרת לא יכול לחפש, אז הדפדפן מוריד הכול כדי להציג עשר שורות ומסנן ב"מכיל" פשוט. חיפוש חכם יותר (רבים, שאלה, משמעות) לא יכול להתקיים כשהרשימה מסוננת בדפדפן, והתשובה גדלה עם כל הוראה חדשה.	החיפוש הוא SQL בשרת. הרשימה מחזירה 9 שדות למסמך, והמסמך המלא נשלח רק להוראה שנפתחת.
2. אין מטמון	אין `Cache-Control` ואין `ETag`, וה-CDN מדווח `EXPIRED`. בביקור אחד אותה רשימה של 75KB ירדה פעמיים: בדף הבית, ושוב במעבר לרשימה.	הנתונים משתנים לעיתים רחוקות, ובכל זאת כל דף, כל ביקור וכל אדם משלמים את ההורדה המלאה. בטלפון או ברשת איטית זה זמן המתנה על משהו שכבר התקבל.	הוספתי מטמון בצד הלקוח (`TanStack Query`): רשימה או מסמך שכבר התבקשו לא מתבקשים שוב 5 דקות, והפרקים מתבקשים פעם אחת בביקור. הנתונים עצמם מתעדכנים פעם ביום במסד שלנו.
3. שלוש קריאות בשרשרת כדי לפתוח הוראה אחת	בחירת הוראה קוראת ל-`DocumentVersion/<id>`, אחר כך ל-`RelatedDocumentId/<id>`, ואחר כך ל-`DocumentAttached/<id>`. כל אחת מתחילה רק אחרי הקודמת (בערך 80ms ביניהן), וכל תשובה קטנה (300 בתים עד 1.3KB).	שלוש התשובות שייכות לאותו מסמך, והמזהה שלו כבר בידי הדפדפן מהרשימה הגדולה. שום קריאה לא תלויה בקודמתה, אז אפשר להחזיר הכול בתשובה אחת. המחיר הוא שלושה מעברי רשת, לא הגודל, וחלונית הפרטים מחכה לשרשרת שלא היה צריך.	קריאה אחת להוראה (`GET /v1/documents/:codeText`) מחזירה את המסמך ואת המסמכים הקשורים (הודעות, טפסים, הוראות זמניות) מ-SQL אחד על הטבלה שלנו. רק הורדת הקובץ פונה לאתר תכ"ם, ורק בלחיצה על הקישור.
4. קריאות שהתשובה שלהן ריקה	`/api/Tag` נקרא בכל טעינה ועונה `[]` (2 בתים), כי לאף מסמך אין תגית. גם `RelatedDocumentId` ו-`DocumentAttached` עונים לעיתים קרובות `[]`.	כל קריאה כזו היא מעבר רשת מלא, עם הכותרות שלו, בלי שום דבר להציג. על המסך יש מסנן תגיות שאין מאחוריו נתונים.	אין קריאת תגיות ואין מסנן תגיות. המסמכים הקשורים הם חלק מהתשובה של המסמך עצמו, כך שהוראה בלי קשורים לא עולה בקשה נוספת.

3 הבעיה המרכזית שבחרתי לפתור: החיפוש

האתגר: אדם עם מצב קונקרטי, למשל "מה עושים אם חרגנו מהתקציב?", לא מצליח להגיע בוודאות להוראה שחלה עליו. בקוד האתר (`filterByFreeTxt`) הטקסט החופשי מושווה לארבעה שדות בלבד באמצעות "מכיל" פשוט: מילה חייבת להיכתב בדיוק כמו בכותרת.

מה מקלידים	מה האתר עושה
קופת ביטחון, ואחר כך קופות ביטחון	2 תוצאות ואז 4: היחיד לא מוצא את הרבים

מה מקלידים	מה האתר עושה
התחייבות, ואחר כך התחייבויות	11 תוצאות ואז 5
מקדמות ללא התחייבות	0 תוצאות, למרות שההוראות על מקדמות (1.4.5) ועל תשלומים ללא התחייבות (1.5.1) קיימות
מה עושים כשיש חריגה	0 תוצאות: שאלה היא לא כותרת
כל חיפוש בלי תוצאה	הכתובת הופכת ל-`/document/undefined` והדף נשאר ריק

על מי זה משפיע: עובדי משרדי הממשלה שחייבים ליישם את ההוראות. **למה כדאי לפתור:** זו הדרך שבה כמעט כל משתמש מגיע להוראה, ואם החיפוש לא מוצא, ההוראה הנכונה פשוט לא מגיעה אליו, ואז הוא מבזבז זמן או מיישם הוראה לא נכונה. (לא מדדתי את ההשפעה על משתמשים אמיתיים: זו השערה סבירה, לא מדידה.)

4 הפתרון

מסד נתונים מקביל, ו-API חיפוש אמיתי מעליו. האובייקטים של האתר מגיעים מ-`Salesforce` (המזהים והשדות שלהם הם של Salesforce: `sfid` , `rtn_*__c`). לדעתי Salesforce הוא מערכת רישום ולא backend של חיפוש: הוא בנוי לקרוא רשומות, לא לחיפוש מדורג, מודע לעברית או סמנטי, והוא מגיע עם מגבלות API. (זו הערכה שלי, לא מדידה של Salesforce.) האתר עצמו מראה את התוצאה: ה-API שלו מוסר לדפדפן את כל הרשימה והדפדפן מסנן. לכן העתקתי את האובייקטים ל**מסד `PostgreSQL` בבעלותנו, מותאם לקריאה**, ו-API ב-`NestJS` שעונה בשאילתות SQL שנכתבו למשימה הזאת בלבד.

חיפוש SQL מדויק על השדות שיש לנו. אותם מסמכים, אבל החיפוש בשרת, על עמודות מנוקות:

- **יחיד ורבים הם אותה מילה;** גרשיים בתוך מילה לא משנים (דו"ח = דוח); מילות שאלה (מה, האם, כשיש) מתעלמים מהן.
- **כל המילים חייבות להופיע. אם אין מסמך כזה, מוצגים הקרובים ביותר והתשובה אומרת זאת.** שאלה לא נגמרת בדף ריק.
- **דירוג:** מילה בכותרת או במספר שווה יותר ממילה בתיאור. במסך רחב ההוראה הטובה ביותר נפתחת מיד.
- **סינון בשרת:** פרק, תת-פרק, סוג וגם תוקף, שמחושב מהתאריכים ולא מהדגל ההפוך.
- **"לא נמצא" היא תשובה תקינה,** עם כתובת תקינה וכפתור לניקוי.
- **מהירות:** שאילתת SQL אחת על עמודות מנוקות מראש. שום רשימה לא נשלחת לדפדפן כדי שיסנן אותה.

5 ה-AI: חיפוש לפי משמעות, בכפתור

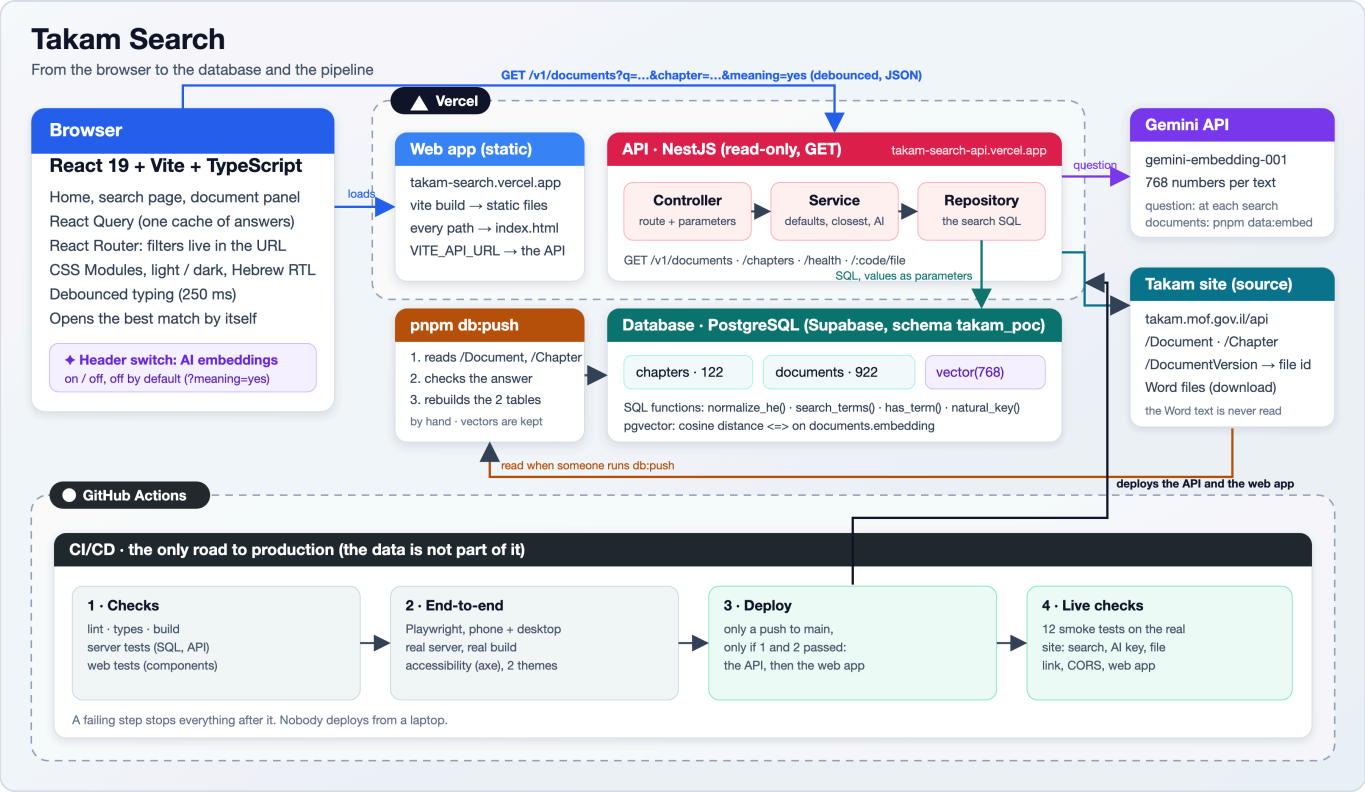
✦ כפתור בכותרת העליונה: מדליקים וכבים את חיפוש ה-AI

הכפתור כבוי כברירת מחדל, ואז החיפוש הוא החיפוש במילים. כשמדליקים אותו, השאלה הופכת ל-`embedding` (וקטור של 768 מספרים, מ-`Gemini`) והמסד מסדר את המסמכים לפי הקרבה למשמעות, בערבוב עם המילים: מספר הוראה ומונח מדויק עדיין מנצחים.

- **איך נוצרים הווקטורים:** סקריפט (`pnpm data:embed`) שרץ ומחשב embedding לכל מסמך מהכותרת, התיאור ומילות המפתח, ושומר אותו בעמודה מסוג `pgvector` (`documents.embedding`) לכל 922 המסמכים. **הווקטורים יושבים במסד הנתונים בלבד**, לא ב-git: הסקריפט מחשב רק למסמכים שאין להם וקטור, וטעינה מחדש של הנתונים משאירה את הווקטורים הקיימים.
- **איך השאלה פוגשת את המסמכים:** שניהם עוברים באותו מודל (`gemini-embedding-001`, 768 מספרים), ולכן הם באותו "מרחב משמעות". המסמכים מחושבים פעם אחת מראש, והשאלה בכל חיפוש. המסד מודד קרבה (`cosine`, האופרטור `<=>` של `pgvector`) ומחזיר את הקרובים ביותר, גם בלי מילה משותפת.
- **דוגמה:** "אפשר לשלם לספק לפני שקיבלתי את הסחורה". החיפוש במילים מחזיר הוראות על רכב ועל מלונות. החיפוש לפי משמעות מעלה את ההוראות 1.4.2 ו-1.4.5 (תשלום מקדמות), בלי שמילה מהשאלה מופיעה בכותרת.
- **בטוח לכבות:** בלי מפתח `Gemini`, או כש-`Gemini` נכשל או שהמכסה היומית נגמרה, החיפוש חוזר אוטומטית לחיפוש במילים והדף אומר זאת (בלי שגיאה). ה-AI יושב מאחורי מחלקה אחת (`AiService`) וכל ספק הוא מחלקה נפרדת.

השלב הבא, אם יש צורך: ליצור וקטורים גם מ**תוכן המסמכים** עצמם (קובצי ה-Word, בחלקים) באותה עמודת `pgvector`, ולתת ל-AI לכתוב תשובה עם הפניה למקורות (`RAG`). זה כבר חיפוש מעמיק יותר, עם עלויות וסיכונים משלו (פענוח קבצים, גודל, תשובה שגויה בהקשר משפטי), ולכן השארתי אותו מחוץ ל-POC: כרגע החיפוש מתבסס על נתוני העל בלבד.

6 הארכיטקטורה: monorepo עם React, Vite, NestJS ומסד נתונים



מהדפדפן ועד מסד הנתונים, כולל הייבוא וה-CI/CD (שמות טכניים באנגלית)

חבילה	מה יש בה
`apps/web`	**הלקוח:** `React 19 + Vite + TypeScript` , `React Query` , `React Router` (הסינון חי בכתובת, כך שאפשר לשתף חיפוש), `CSS Modules` , עברית מימין לשמאל, מצב בהיר וכהה. מבנה הדפים כמו באתר המקורי.
`apps/server`	**השרת:** `NestJS` , תיקייה לכל route, ובכל אחת `controller → service → repository` , ותיקיית `ai/` ל-AI. קריאה בלבד (`GET`).
`packages/contracts`	הטיפוסים המשותפים ללקוח ולשרת: צד אחד משתנה בלי השני, בדיקת הטיפוסים נכשלת.

- **סינון פרקים עם חיפוש בתוך הרשימה:** שדות הפרק הראשי והמשני אפשר להקליד בהם, כמו באתר המקורי: הרשימה מצטמצמת לפי מה שמקלידים (מספר או מילים), עם ניווט במקלדת (חיצים, `Enter` , `Escape`) ועם תבנית `ARIA combobox` נגישה.
- **הלקוח קורא ל-API דרך מטמון (`TanStack Query`), כפי שמומלץ בתיעוד שלהם:** מצב השרת יושב במטמון ייעודי אחד ולא ב- `useState` ו- `useEffect` ידניים. כל תשובה נשמרת לפי המפתח שלה (המסננים בחיפוש, מספר ההוראה), ולכן חזרה לחיפוש או להוראה שכבר נראו מוצגת מיד, בלי בקשה. תשובה נשארת "טרייה" 5 דקות, והפרקים לא נטענים שוב במהלך הביקור. בקשות זהות מתאחדות לבקשה אחת, בקשה ישנה מתבטלת כשהמשתמש ממשיך להקליד (יחד עם השהיה של 250 מילישניות), והרשימה הקודמת נשארת על המסך עד שהבאה מגיעה (`keepPreviousData`), כך שאין הבהוב של דף ריק. כשל בשרת נוסה שוב פעמיים, סירוב (4xx) לא, ומצבי הטעינה, השגיאה וה"ניסיון חוזר" באים מאותו מקור. התוצאה: ממשק מהיר ואינטואיטיבי, ופחות עומס על ה-API.
- **מסד הנתונים:** `PostgreSQL` ב- `Supabase` (סכמה `takam_poc` , עם התוסף `pgvector`), שה-API קורא ממנו בעסקאות לקריאה בלבד. טבלאות `chapters` ו- `documents` ופונקציות SQL שמנקות עברית. בלי

`DATABASE_URL` (למשל אצל מי שמריץ מקומית) ה-API בונה לעצמו `PostgreSQL` בתוך התהליך (`PGlite`) וממלא אותו מאתר תכ"ם בהפעלה הראשונה. אין קבצי נתונים ב-git: הבדיקות קוראות עותק שמור של שתי תשובות האתר.

- **הכנסת נתונים:** `pnpm db:push` ביד, כשהאתר השתנה: קורא את `/Document` ו-`/Chapter`, בודק שהתשובה נראית תקינה, ובונה מחדש את שתי הטבלאות בעסקה אחת (הווקטורים הקיימים נשמרים). אין משימה יומית.
- **הכללים הם חלק מהארכיטקטורה:** הם כתובים ב-`CLAUDE.md` ונאכפים בבדיקות ובצנרת: שכבות השרת וכיוון התלויות, החיפוש תמיד ב-SQL עם ערכים כפרמטרים, API לקריאה בלבד עם גוף שגיאה אחד, ה-AI רק מאחורי `AiService` (הבדיקות אף פעם לא קוראות ל-AI אמיתי), כל טקסט הממשק רק ב-`messages.ts`, כל הצבעים רק ב-`tokens.css` (ניגודיות AA בשני המצבים, נבדקת), מצב החיפוש בכתובת, והתיעוד מתעדכן באותו commit כמו הקוד.

7 בדיקות ואיכות

רמה	מה נבדק	כמות
יחידה ו-API בשרת	פונקציות ה-SQL, הטעינה, שאילתת החיפוש על הנתונים האמיתיים בזיכרון, השירות, מודול ה-AI (ספק מזויף ו-`fetch` מזויף), הלקוח של אתר תכ"ם, וכל route מקצה לקצה	147
יחידה ורכיבים בלקוח	דפים מול API מזויף, כל רכיב, לקוח ה-API, טוקני העיצוב (ניגודיות AA)	99
קצה-לקצה (`Playwright`)	טלפון ושולחן עבודה, שרת אמיתי ו-build אמיתי, נגישות (`axe`) בשני המצבים	108
האתר החי	`smoke tests` אחרי כל פריסה	12

> **כיסוי, נמדד ולא מושלם:** שרת 99.3% הצהרות, 89.0% ענפים, 100% פונקציות ושורות; לקוח 98.5% הצהרות, 93.8% ענפים, 97.8% פונקציות, 99.3% שורות (בדיקות הקצה-לקצה לא נספרות). הצנרת נכשלת אם שינוי מוריד את הכיסוי מתחת לרף. לא כתבתי "כיסוי מושלם" כי הוא לא כזה.

מקרי קצה שנבדקים: תשובה ריקה, חיפוש של מילה אחת ושל שאלה, `H.7.1.5` (שהאתר מסתיר), פרק 10 (שחסר באתר), יחיד ורבים, גרשיים, ניסיון הזרקה במילים, מסמך שלא קיים, מפתח AI חסר או AI שנכשל, אתר תכ"ם שנכשל, מזהה קובץ שגוי, CORS לקריאה בלבד, ומסך טלפון ושולחן עבודה.

צנרת אחת, ורק GitHub פורס: בדיקות (`lint`, טיפוסים, `build`, יחידה וכיסוי) ← קצה-לקצה ← פריסה ל-`Vercel` (רק מ-`main` ורק אחרי שהכול עבר) ← 12 בדיקות על האתר החי. אף אחד לא פורס ממחשב אישי.

8 חלק ג: חשיבה הנדסית והתאמה ל-Production

1. תכנון והחלטות הנדסיות

השיקולים: החיפוש הוא המוצר, וכל השאר משרת אותו; הנתונים נמצאים ב-Salesforce שהוא מערכת רישום, ולכן מסד מקביל בבעלותנו; שרת פשוט שאפשר לעקוב בו אחרי בקשה בארבעה קבצים; ומה שמסוכן (ה-AI, אתר תכ"ם) יושב מאחורי מחלקה אחת ומתג. **חלופות שבדקתי:** חיפוש ב-Salesforce עצמו או סינון ברשימה המלאה בדפדפן; `Elasticsearch` (מערכת שנייה בשביל 922 מסמכים); `SQLite` או מסד מנוהל; `RAG` מהתחלה. **על מה ויתרתי ב-POC:** תוכן המסמכים ותשובה שה-AI כותב, מסד משותף ועמיד, דפדוף בלקוח, הרשאות, מילון עברי (הכללים כתובים ביד) ותגיות.

2. איכות ותחזוקתיות: מגבלה משמעותית

הנתונים נכנסים למסד ביד (`pnpm db:push`) ואין עדכון אוטומטי. ההשלכות: המסמכים עדכניים עד ההכנסה האחרונה; הווקטור של מסמך ששונה נשאר כמו שהיה, ולמסמך חדש אין וקטור עד `pnpm data:embed` (החיפוש במילים עדיין מוצא אותו); המסד ב- `Supabase` משותף לאפליקציות נוספות, ולכן הטבלאות שלנו בסכמה נפרדת בלבד, אבל הפרויקט והמשתמש אינם שלנו לבד; ובלי `DATABASE_URL` המסד נבנה מחדש בכל הפעלה (8 עד 10 שניות ב-Vercel). וכללי העברית וחלון הקרבה של חיפוש ה-AI כוונו ידנית על מדגם קטן. **השינוי שהייתי עושה:** משימה מתוזמנת שמריצה את `db:push` ו- `data:embed` ומודיעה כשהתשובה נדחית, משתמש משלנו עם הרשאות רק על `takam_poc` , ותאריך ההכנסה האחרונה ב- `/health` . ובנוסף: אוסף שאלות אמיתיות עם ההוראה הצפויה כדי למדוד את החיפוש.

3. תהליכי פיתוח ופריסה

מה יקשה על מפתח אחר היום: שני פרויקטי Vercel, משתני הסביבה שלהם (`CORS_ORIGINS` , `VITE_API_URL` , `GEMINI_API_KEY`) וסוד ומשתנים ב-GitHub נוצרו ביד; ה-API נבנה בתוך `apps/server` ונפרס מהשורש כי נתיבי ה-build יחסיים לשורש; קובצי ה-binary של PGlite נקראים בנתיבים מפורשים כדי ש-Vercel יכלול אותם; והכול קשור ל-Vercel. **מה כבר קיים:** `pnpm install && pnpm dev` מריץ הכול בלי סוד, `.env.example` מפרט כל משתנה, והצנרת בודקת, פורסת ובודקת את האתר החי. **מה הייתי משנה:** תשתית כקוד (פרויקטי Vercel ומשתניהם במאגר, או `Dockerfile` שמתאים לכל מארח), סקריפט `bootstrap` שיוצר סביבה חדשה ומדפיס מה חסר, וגרסאות נעולות. **איך מוודאים אחרי פריסה:** 12 ה- `smoke tests` בשלב האחרון של הצנרת, ו- `/health` שמדווח כמה מסמכים ופרקים נטענו.

4. אבחון וטיפול בתקלות

תרחיש: אתר תכ"ם משנה את מבנה הנתונים או שהוא למטה כשקוראים אותו. **איך זה היה מתבטא:** בלי הגנה, המסד היה נטען עם מסמכים בלי מספר, חיפוש לפי מספר לא היה מוצא כלום וה-API עדיין היה נראה תקין. **איך מזהים ומטפלים היום:** `pnpm db:push` בודק את התשובה (חייבת להיות רשימה, עם לפחות 900 מסמכים שיש להם מספר ו-100 פרקים) לפני שהוא נוגע במסד; אם התשובה לא תקינה או שהאתר למטה, הוא עוצר והמסד **שומר את הנתונים שיש לו**. הטבלאות נבנות מחדש בעסקה אחת: כשל משאיר את הנתונים הישנים, וה-API אף פעם לא רואה מסד חצי מלא. ה- `/health` מדווח כמה מסמכים ופרקים נטענו, וה-smoke tests מכשילים את הצנרת אם הנתונים החיים לא נטענו. **מה הייתי משנה:** הרצה מתוזמנת עם הודעה כשהתשובה נדחית, ותאריך ההכנסה האחרונה שהצליחה ב- `/health` . **תרחיש נוסף:** `Gemini` נכשל או שהמכסה נגמרה: חיפוש ה-AI חוזר לחיפוש במילים והדף אומר שה-AI לא זמין (בלי שגיאה), והכשל נרשם ב- `AiService` . **זה קרה באמת ב-11.11.2026:** המכסה היומית החינמית של ה- `embeddings` נגמרה אחרי שחישבתי את 922 המסמכים, והתיקון היה החזרה האוטומטית הזאת.

9 הרצה, קונפיגורציה והדגמה

- **הרצה:** `pnpm install` ואז `pnpm dev` (דרישות: `Node ≥ 22` ו- `pnpm 11`). האתר ב- `localhost:5173` וה-API ב- `localhost:3100` . אין צורך בסוד כדי להריץ.
- **חיפוש ה-AI:** מפתח `Gemini` ב- `apps/server/.env` (`git` מתעלם ממנו), כפי שמופיע ב- `.env.example` . אף סוד לא נמצא בקוד.
- **בדיקות:** `pnpm test` , `pnpm test:coverage` ו- `pnpm test:e2e` .
- **נתונים:** נקראים מאתר תכ"ם (`pnpm db:push`); עותק שמור של שתי התשובות לבדיקות ב- `apps/server/test/fixtures/` .
- **סרטון הדגמה (עד 3 דקות):** מציג את החיפוש במילים (יחיד ורבים, שאלה, "לא נמצא"), ואז מדליק את כפתור ה-AI ומראה את השאלה שמוצאת מה שהחיפוש במילים לא מצא.